

Stefan Miruna Andreea 334 CA

Tema 1 IA

Rezolvarea jocului Sokoban folosind LRTA* si Beam Search

Cuprins

1. Introducere
2. LRTA*
 - 2.1. Prima euristica - bazata pe Manhattan Distance
 - 2.2. A doua euristica - bazata pe Hungarian Algorithm
3. Beam Search
 - 3.1. Comportamentul primei euristici pe Beam Search
 - 3.2. Comportamentul celei de-a doua euristici pe Beam Search
4. Concluzii: Comparatii LRTA* - Beam search
5. Resurse

1. Introducere

Scopul acestui document este de a jurnaliza ideile care au dus la construcția celor 2 euristici pe care le-am folosit pentru rezolvarea jocului Sokoban cu ajutorul algoritmilor LRTA* și Beam Search, precum și optimizarile aduse acestor doi algoritmi. De asemenea, se urmărește măsurarea calității euristiciilor, dar și comparația dintre cei doi algoritmi pe baza unor metrici precum: timpii de execuție, numărul de stări construite și numărul de mișcări de pull (care sunt un indicator pentru calitatea soluției).

2. LRTA*

2.1. Prima euristica - bazata pe Manhattan Distance

Ideea de început - Distanța Manhattan

Dat fiind faptul ca jucătorul se poate muta doar sus-jos, respectiv stanga-dreapta, am ales sa pornesc de la distanța Manhattan. Mai exact, am inceput de la premisa ca valoarea euristicii unei stari ar trebui sa fie egala cu suma distanțelor manhattan minime de la fiecare cutie pana la target-ul cel mai apropiat de ea.

Modificarea funcției de cost din curs

În funcția de cost, am considerat ca toate acțiunile care duc la trecerea dintr-o stare în alta au același cost, mai exact 1. Deși în curs costul pentru o stare care nu mai fusese încă întâlnită se calcula ca $h(s_{prev})$, am ales sa folosesc $h(s)$ (unde s = starea curentă), deoarece, ruland pe prima hartă (easy_map1, in care avem o singura cutie, respectiv un singur target), am observat ca se faceau unele mutari care îmi indepartau cutia de țintă în loc să o apropie. Acest lucru se intampla pentru ca de fiecare data cand exploram o stare noua intram pe prima ramura a if-ului si intorceam mereu distanta manhattan a stării de

dinainte, adică luam în considerare mereu aceeași distanță, fără să ne dăm seama că ne îndepărtăm de fapt de tinta. Aceasta situație este prezentată vizual în figura 1.

Funcția $\text{Cost}(s_{\text{prev}}, a, s)$ întoarce o estimare de cost
dacă s nu este definită **atunci întoarce** $h(s_{\text{prev}})$
altfel întoarce $\text{cost}(s_{\text{prev}}, a, s) + H[s]$

Curs 3 - Cautari locale, pag.28

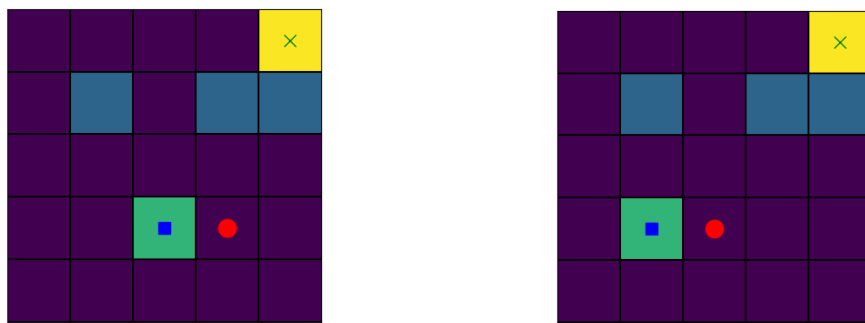


Figura 1

```
def cost(self, s_prev, s):
    if str(s) not in self.H:
        return 1 + manhattan_distance(s_prev)
    return 1 + self.H[str(s)]
```

```
def cost(self, s_prev, s):
    if str(s) not in self.H:
        return 1 + manhattan_distance(s)
    return 1 + self.H[str(s)]
```

Modificarea în cod a funcției de cost

Facând această modificare, am redus numărul de stări explorate pentru rezolvarea primei harti (easy_map1) de la 607 la 91.

Penalizarea stărilor deja vizitate

Urmărind gif-ul generat pentru aceasta prima harta, acum am observat o nouă problema: jucătorul trece inutil de mai multe ori prin aceleași poziții despre care știm deja ca nu conduc spre goal state. Urmând o tehnica din reinforcement learning, bazată pe recompense si penalizari, am încercat sa adaug un cost mai mare de întoarcere in stările pe care deja le-am vizitat pentru a împiedica agentul sa se intoarca si sa piardă timp cu revizitarea acestora și pentru a încuraja explorarea de stări noi, motiv pentru care în loc sa adun tot 1 la h-ul anterior, am adunat 60 pentru stările deja vizitate (am ales 60 prin încercări, acest număr ar putea fi ajustat în funcție de dimensiunea hărții). Astfel, numărul de mișcări s-a redus de la 91 la 79, iar timpul de rulare a scăzut de la 0.0135 secunde la 0.0110 secunde. Articolul despre reinforcement learning din care m-am inspirat este [1].

Acum ca am făcut si aceasta ajustare, este prima oară cand algoritmul Irta* reușește să găsească soluții pentru toate hărțile, după cum se observa în screenshot-ul de mai jos:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0241	79	1
easy_map2.yaml	True	0.0152	17	0
medium_map1.yaml	True	0.0248	65	4
medium_map2.yaml	True	0.1966	947	58
large_map1.yaml	True	0.0600	169	1
large_map2.yaml	True	5.7975	19172	485
hard_map1.yaml	True	0.2497	1213	75
hard_map2.yaml	True	0.1943	1030	31

Pentru a îmbunătăți euristica pe care am construit-o până în acest punct, m-am inspirat din teza de doctorat a lui Andreas Junghanns intitulată "Pushing the Limits: New Developments In Single-Agent Search" [2] și din teza de masterat a lui Timo Virkkala, intitulată "Solving Sokoban" [3].

Evitarea deadlock-urilor

Ambii vorbesc despre o noțiune importantă, mai exact cea de **dead positions / deadlocks** (Vikkala - Capitolul 4.3.1: "A position in a Sokoban puzzle is called dead if a stone pushed into it can never reach any goal.", Junghans - Capitolul 3.1.2: "If a stone is pushed into a corner, it is permanently immobilized, and can never be pushed to a goal. Therefore, the problem becomes unsolvable."). Un deadlock este o poziție din care cutia nu mai poate fi împinsă către destinație (fara o miscare de pull), mai exact dacă ajunge sa aiba 2 margini care se intersectează blocate, adică să ajungă într-un colț al tablei de joc sau într-un colț cu obstacolele de pe tabla de joc.

Am implementat funcția de `would_be_dead_pos()`, care întoarce True dacă poziția în care s-ar muta cutia ar fi una "dead" (asta în cazul în care nu cumva target-ul însuși se afla într-un astfel de colț, caz în care se întoarce False) și am asociat o penalizare infinit de mare pentru aceste poziții pentru a evita ajungerea în deadlock-uri. După aceasta imbunatatire, toate hartile au putut fi rezolvate cu succes într-un număr de pași considerabil mai mic (dacă facem media tuturor hartilor), după cum se poate observa în imaginea de mai jos:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0160	39	0
easy_map2.yaml	True	0.0141	17	0
medium_map1.yaml	True	0.0236	65	4
medium_map2.yaml	True	0.1034	434	26
large_map1.yaml	True	0.1601	563	17
large_map2.yaml	True	1.4475	4757	119
hard_map1.yaml	True	0.0777	291	25
hard_map2.yaml	True	0.0339	111	1
super_hard_map1.yaml	True	0.2932	1242	64

Introducerea poziției player-ului fata de cutie

Până în momentul de fata, în construcția acestei euristici am luat în considerare doar poziția cutiei fata de target, motiv pentru care player-ul ajunge în unele situații să se plimbe degeaba pe harta, indepartandu-se de cutie, deoarece poziția playerului nu este nicăieri contorizata, euristica folosind mereu distanța de la cutie la target, iar cutia ramanand nemiscata. De aceea, atat Timo Vikkala, cât și Junghans evidențiază faptul că și distanța de la player la cutie trebuie luată în considerare în calculul euristicii. În secțiunea 4.3.5 (“Position of the Man”) din teza sa, Junghans spune: “Simply using the distance of the stone to the goals ignores an important issue: The position of the man with respect to the stone to be pushed. What is the distance of a stone to a particular goal? One could assume the man is able to travel from anywhere in the maze to anywhere else. However, the maze, even with only one stone in it, restricts the man’s movements. If a stone’s path leads through an articulation point in the maze, the man’s movement is restricted by that one stone.”.

Astfel, la costul total am adaugat și distanța manhattan de la player la fiecare cutie care nu se afla pe target-ul sau, căci și aceasta conteaza, chiar dacă nu într-o maniera la fel de importantă ca distanța de la cutie la target (de aceea am și înmulțit-o cu un factor subunitar). Rezultatele au fost următoarele:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0166	41	2
easy_map2.yaml	True	0.0135	17	0
medium_map1.yaml	True	0.0150	15	5
medium_map2.yaml	True	0.0690	226	22
large_map1.yaml	True	0.0708	206	12
large_map2.yaml	True	0.3145	923	57
hard_map1.yaml	True	0.1296	498	36
hard_map2.yaml	True	0.2653	1253	71
super_hard_map1.yaml	True	0.9096	2771	486

De asemenea, am observat ca în hărțile cu mai multe cutii (respectiv mai multe target-uri), uneori playerul muta niște cutii, chiar dacă acestea ajunsesera sa fie positionate corect, pe target-urile corespunzătoare. De aceea, am mai facut următoarea îmbunătățire în euristica: dacă ajungem la o cutie care este deja pe target, o ignorăm și trecem la următoarea, căci distanța de la ea la țintă este nulă. Acum rezultatele se prezinta asa:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0162	41	2
easy_map2.yaml	True	0.0128	17	0
medium_map1.yaml	True	0.0136	15	5
medium_map2.yaml	True	0.1669	714	53
large_map1.yaml	True	0.0471	120	0
large_map2.yaml	True	0.0357	58	5
hard_map1.yaml	True	0.0282	58	4
hard_map2.yaml	True	0.0645	234	21
super_hard_map1.yaml	True	0.1522	540	51

Pull Positions

În urma adăugării distanței manhattan de la player la cutie și a îmbunătățirii anterioare, deși se observa overall o scădere a numărului de stări vizitate pentru a ajunge la soluție, în unele cazuri și numărul de pull-uri a crescut. Acest lucru se întâmpla pentru că noi luăm în considerare doar distanța de la player la cutie, nu și poziția în care se va afla playerul raportat la cutie și la direcția ei de deplasare către target. De exemplu, să luăm 2 situații în care playerul se afla la distanța 1 față de cutie (adică se afla fix în casuta vecină). Să zicem că target-ul se afla undeva în dreapta cutiei, pe aceeași linie cu ea. Dacă playerul se afla pe poziția vecină din dreapta, atunci va fi obligat să facă o mișcare de pull pentru a trage cutia către tinta, lucru care noua nu ne convine și ar trebui penalizat. Dacă playerul se afla pe poziția vecină din stânga, el va trebui doar să împingă cutia până la target, deci nu va fi nevoit să facă nicio mișcare de pull. Iată cum, deși distanța de la player la cutie este aceeași, poziția lui față de cutie (raportat și la direcția către target) face diferența. Din acest motiv am creat funcția `is_pull_position()`, care determină direcția în care trebuie să o ia cutia pentru a ajunge la target și verifică dacă playerul se interpune între cutie și target (ceea ce ar duce la o mișcare de pull) sau dacă se învecinează cu box-ul în direcția opusă (ceea ce ar fi cazul ideal, pentru că nu ar implica decât mișcări de push). Dacă această funcție întoarce `true`, înseamnă că starea aceasta ar implica o mișcare de pull, pe care va trebui să o penalizăm, pentru a descuraja cât mai mult ajungerea în astfel de situații. Pentru fiecare pull, adun 30 la distanța totală. (Și în acest caz tot empiric am ales valoarea 30, suficientă ca să facă o diferență, dar nici prea mult ca să ajungă să devină ineficient pentru că playerul se tot învârtă până când ajunge să se hotărască dacă trebuie neapărat să facă pull sau nu.

Chiar dacă în unele cazuri numărul de stări a crescut (ceea ce este și normal), numărul de pull-uri s-a redus considerabil). Rezultatele obținute în urma penalizării pull-urilor:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0207	45	2
easy_map2.yaml	True	0.0174	17	0
medium_map1.yaml	True	0.0443	121	5
medium_map2.yaml	True	0.0503	150	5
large_map1.yaml	True	0.0577	142	0
large_map2.yaml	True	0.1127	306	5
hard_map1.yaml	True	0.0711	237	13
hard_map2.yaml	True	0.0480	139	8
super_hard_map1.yaml	True	0.3977	1545	97

2.2. A doua euristica - bazata pe Hungarian Algorithm

Pentru cea de-a doua euristica, in loc sa pornesc de la distantele manhattan minime de la cutii la target-urile cele mai apropiate, cum am facut la prima euristica, am pornit de la o idee enuntata de Andreas Junghans in teza sa: cum fiecarei cutii ii corespunde o singura tinta, problema se modeleaza sub forma unui graf bipartit, iar scopul nostru este de a gasi atribuirile optime intre fiecare element din multimea stanga (cutiile) si fiecare element din multimea dreapta (target-urile).

“The fundamental observation leading to the lower-bound heuristic used in *Rolling Stone* is the following: Only one stone can go to any one goal. For each stone, there is a minimum number of pushes required to maneuver that stone to a particular goal. This distance (or cost) assumes no adverse interactions with other stones in the maze, basically pretending the maze is empty. The problem is to find the assignment of stones to goals that minimizes the sum of these distances.

Since there are as many stones as there are goals, and every stone has to be assigned to a goal, we are trying to find a minimum cost (distance) perfect matching on a complete bipartite graph. Edges between stones and goals are weighted by the distance between them, and assigned infinity if the stone cannot reach a goal. We will call this heuristic **Minmatching** for short.” - (capitolul 4.3.1 “Minimum Cost Matching”).

Sa alegem pentru fiecare cutie targetul care se afla la distanta manhattan minima nu este intotdeauna cea mai buna solutie, deoarece s-ar putea ca daca alegem pentru primele cutii targeturile cele mai apropiate, pentru urmatoarele cutii sa ramana disponibile doar niste targeturi extrem de indepartate si sa se ajunga per total la un cost mai mare. Junghans da urmatorul exemplu in teza lui (tot in capitolul 4.3.1):

Daca pentru Cc am alege tinta Bb (care se afla la distanta minima fata de Cc), lui Hb nu i-am mai putea asigna nicio tinta, pentru ca Bb ar fi deja asignat altcuiva. Prin urmare, trebuie sa abordam o viziune care ia in considerare toate cutiile, nu doar cutia curenta. Chiar daca atat Cc, cat si Id au target-uri mai apropiate de ele, va trebui sa le impingem catre target-uri aflate un pic mai departe pentru a ne asigura ca fiecarei cutii ii poate fi asignata o tinta si ca per total costul este cel mai avantajos. Intr-un final se vor alege caile bold-uite. Aceasta metoda de asignare va ajuta la eliminarea unor parti mari din spatiu de cautare.

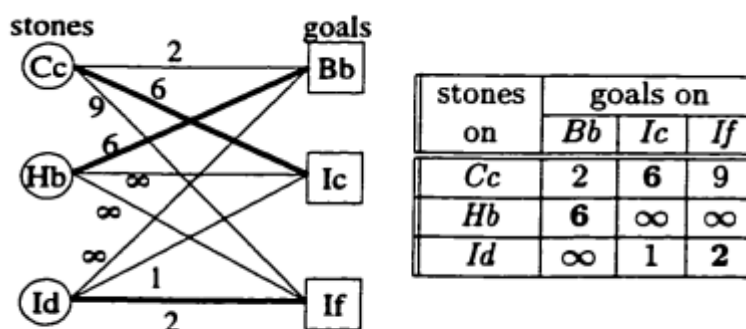
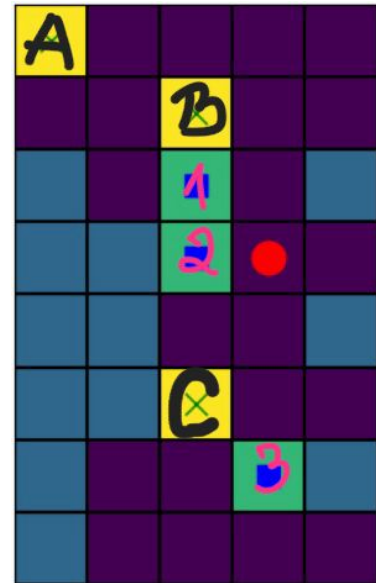


Figure 4.1: Minmatching Example

Concret pentru testele noastre, harta medium_map2 arata asa:

Problema este urmatoarea: cum noi ii asociem fiecarei cutii target-ul aflat la cea mai mica distanta manhattan de cutia respectiva, se va incepe cu box-ul 1, care se va muta pe target-ul B, apoi pentru cutia 2 cea mai apropiata tinta ramasa disponibila va fi C, iar cutia 3 va trebui sa ajunga la singura tinta care a mai ramas disponibila pentru ea, adica A. O asociere mai buna a cutiilor cu targeturile ar fi urmatoarea: 1 - A, 2 - B, 3 - C. Chiar daca tinta B este mai aproape de cutia 1 decat tinta A, per total intr-un final vom constata ca a fost mai rentabil sa asociem 1 cu A decat cu B. Dar aceasta problema este fix motivul pentru care am creat cea de-a doua euristica. Aceasta nu mai face asocierea cutiilor cu targeturile pe baza distantei manhattan minime, ci foloseste hungarian algorithm, al carui scop este de a face asocierea cutie - target intr-un mod optim overall.



Hungarian Algorithm, care rezolva aceasta problema de atribuire optima in graful bipartit, functioneaza in felul urmatoare:

Noi avem multimea de cutii si multimea de targeturi: fiecare cutie trebuie asociata cu un singur target. Mai intai, se construiesc o matrice $(n \times n)$ de costuri, unde fiecare element $cost_matrix[i][j]$ reprezinta distanta manhattan de la cutia i la targetul j . Pe baza acestei matrice, algoritmul va gasi match-ul perfect cu cost total minim, aplicand urmatoorii pasi:

1. Pentru fiecare rand, scade cel mai mic element aflat pe randul respectiv din fiecare element de pe acel rand. Pe pozitia unde se afla elementul cel mai mic vom avea acum 0.
2. Aplica exact acelasi proces pe coloane.
3. Traseaza un numar minim de linii pe randurile si coloanele care contin 0 in asa fel incat toate 0-urile sa fie acoperite.
 - a. Daca am trasat n linii, atunci ne oprim pentru ca am gasit o atribuire optima.
 - b. Daca numarul de linii trasate este mai mic decat n , trecem la pasul urmator pentru ca inseamna ca nu am atins numarul optim de 0-uri.
4. Gaseste cel mai mic element care nu este acoperit de nicio linie. Scade acest element din fiecare rand care NU este taiat si apoi aduna-l la fiecare coloana care este taiata. Apoi, mergi inapoi la pasul 3.

Asignarea corecta se face prin alegerea 0-urilor in asa fel incat nicio linie/coloana să nu conțină mai mult de un zero selectat.

Pasii detaliati si exemplificati ai acestui algoritm sunt explicati pe acest site: [4].

Eu nu am mai implementat manual toti acesti pasi, deoarece am folosit functia `linear_sum_assignment()` din biblioteca `scipy.optimize` (a carei documentatie se gaseste aici: [5]), care primeste ca parametru matricea de costuri si intoarce doi vectori, unul pentru indicii cutiilor si unul pentru indicii target-urilor. Cutiei cu indicele de pe pozitia x din `box_indices` ii va corespunde target-ul cu indicele de pe aceeasi pozitie x din `target_indices`. Se face suma acestor costuri si se afla costul total al asocierii.

Hungarian algorithm + eliminarea deadlock-urilor

Folosind acest algoritim si acea imbunatatire pentru deadlocks pe care am utilizat-o si mai devreme obtinem urmatoarele rezultate:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0180	39	0
easy_map2.yaml	True	0.0151	17	0
medium_map1.yaml	True	0.0267	65	4
medium_map2.yaml	True	0.2324	973	57
large_map1.yaml	True	0.1699	563	17
large_map2.yaml	True	0.7821	2330	42
hard_map1.yaml	True	0.0793	262	17
hard_map2.yaml	True	0.0377	111	1
super_hard_map1.yaml	True	0.0382	84	11

Adaugarea distantei de la player la cutia cea mai apropiata

Conform observatiilor facute la prima euristica, este important sa luam in considerare si distanta de la player la cutie. De data aceasta, vom aduna distanta de la player la cea mai apropiata cutie, caci acum nu mai parcurgem rand pe rand toate cutiile din vector. Acum rezultatele devin urmatoarele:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0196	41	2
easy_map2.yaml	True	0.0147	17	0
medium_map1.yaml	True	0.0230	50	2
medium_map2.yaml	True	0.1051	336	17
large_map1.yaml	True	0.0451	87	5
large_map2.yaml	True	0.3144	844	16
hard_map1.yaml	True	0.0449	98	5
hard_map2.yaml	True	0.0321	80	2
super_hard_map1.yaml	True	0.0473	101	17

Descurajarea miscarilor de pull

Pentru a evita pe cat posibil miscarile de pull, putem aplica penalizari suplimentare (in valoare de 100) in functia de cost a algoritmului Irta* daca observam ca numarul de miscari de pull pentru starea curenta a crescut fata de numarul de miscari de pull din starea anterioara. De asemenea, avand in vedere ca am observat ca in hartile cu mai multe cutii, respectiv target-uri, unele cutii ajunse deja pe targeturile corespunzatoare sunt mutate degeaba (fiind ulterior aduse pe aceeasi pozitie), daca vrem sa descurajam miscarile inutile care implica mutarea cutiilor aflate deja pe targeturile corespunzatoare, putem aplica si pt asta o penalizare. In functie de penalizarea pe care o aplicam obtinem diverse rezultate.

Daca alegem o penalizare de 150 pentru aceasta mutare inutila a cutiilor, obtinem rezultatele urmatoare:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0197	43	0
easy_map2.yaml	True	0.0147	17	0
medium_map1.yaml	True	0.0278	68	0
medium_map2.yaml	True	0.1626	678	7
large_map1.yaml	True	0.0453	98	0
large_map2.yaml	True	0.7373	2148	6
hard_map1.yaml	True	0.1069	380	5
hard_map2.yaml	True	0.0364	107	0
super_hard_map1.yaml	True	0.4079	1385	14

Daca alegem o penalizare de 200 pentru aceasta mutare inutila a cutiilor, obtinem rezultatele urmatoare:

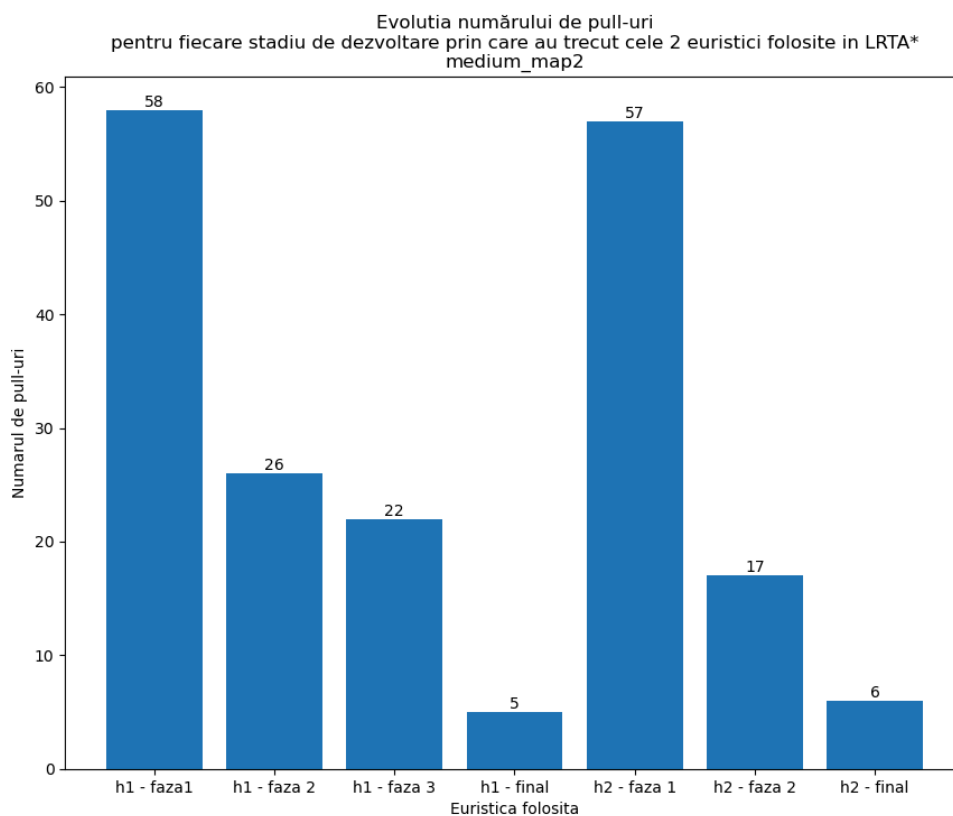
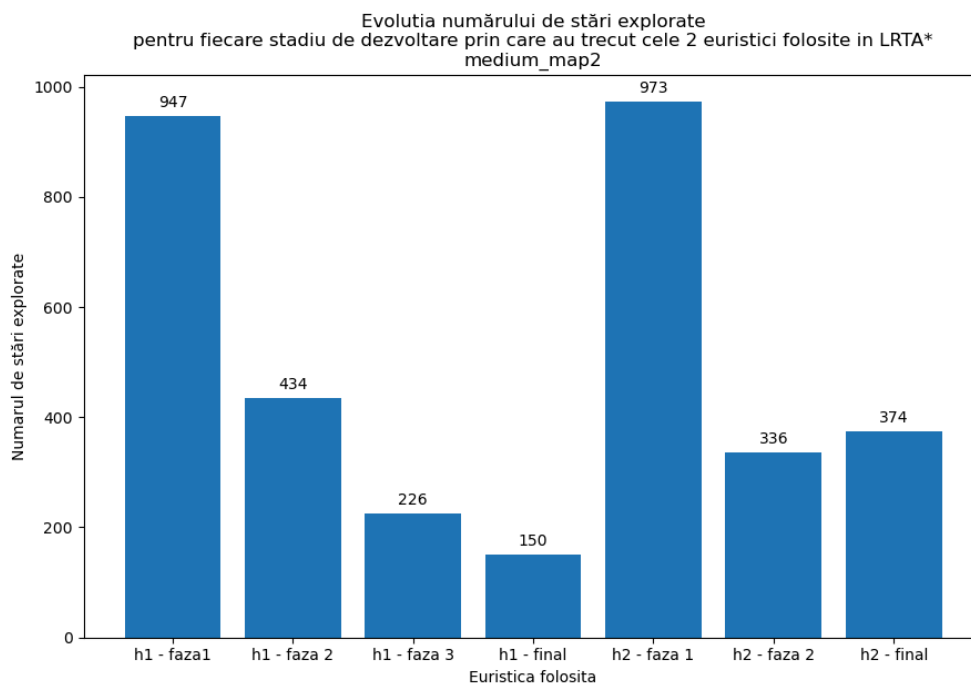
Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0193	43	0
easy_map2.yaml	True	0.0152	17	0
medium_map1.yaml	True	0.0286	68	0
medium_map2.yaml	True	0.0975	374	6
large_map1.yaml	True	0.0460	98	0
large_map2.yaml	True	0.6290	1840	6
hard_map1.yaml	True	0.1077	380	5
hard_map2.yaml	True	0.0364	107	0
super_hard_map1.yaml	True	0.6623	2343	26

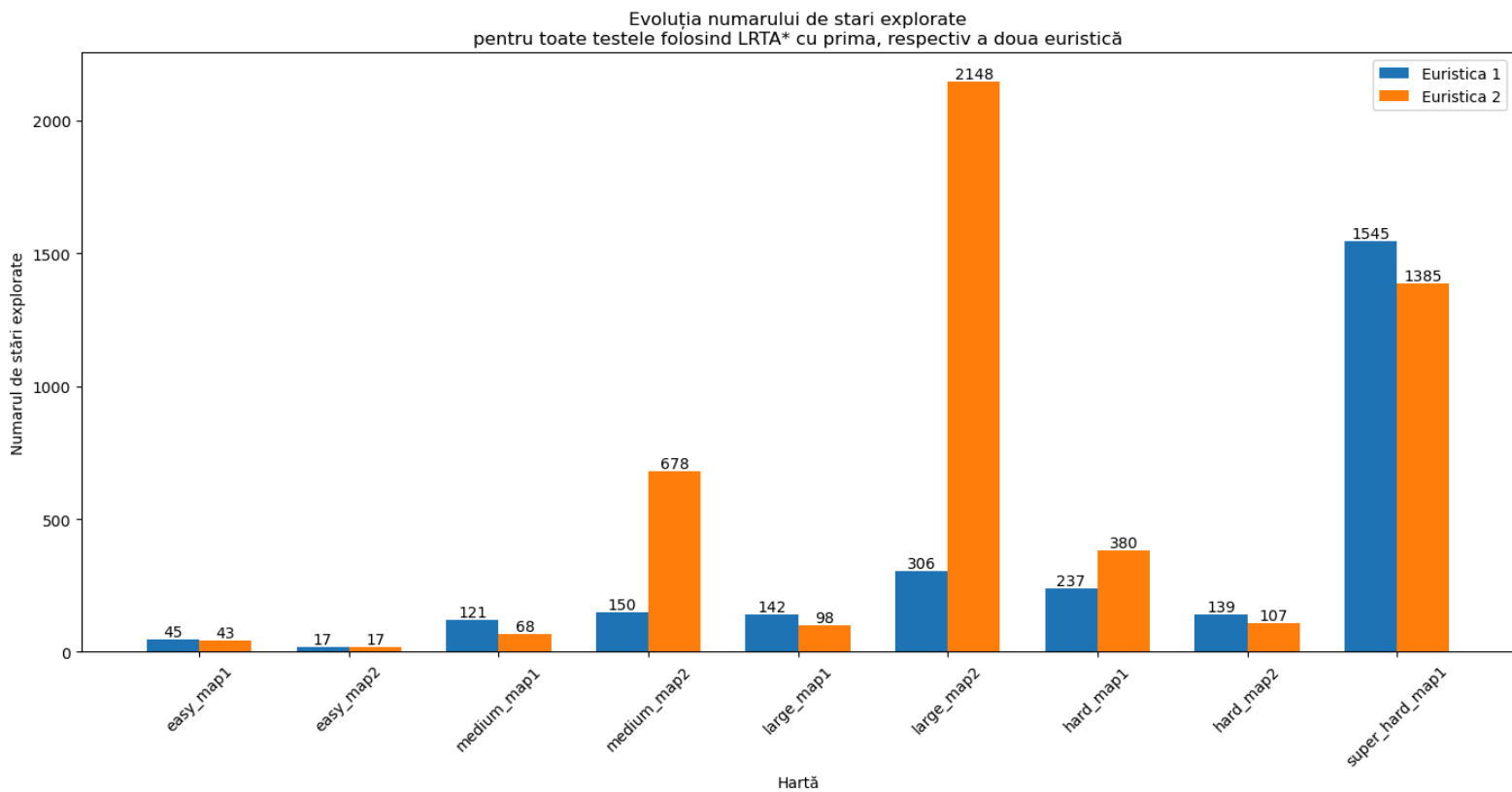
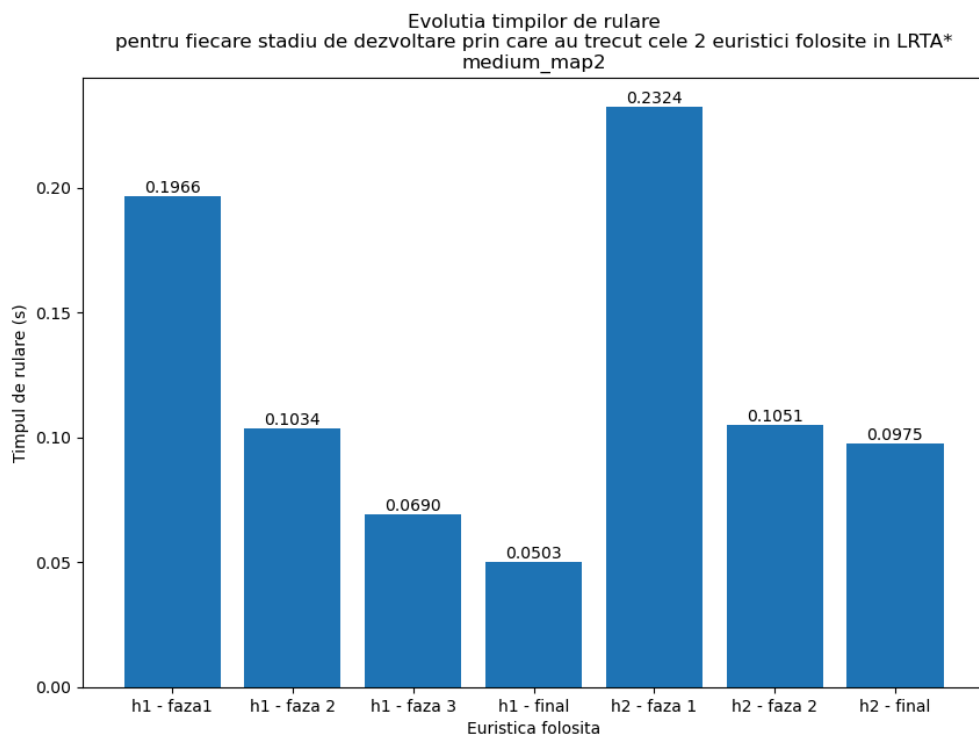
Analiza cantitativa a celor doua euristici

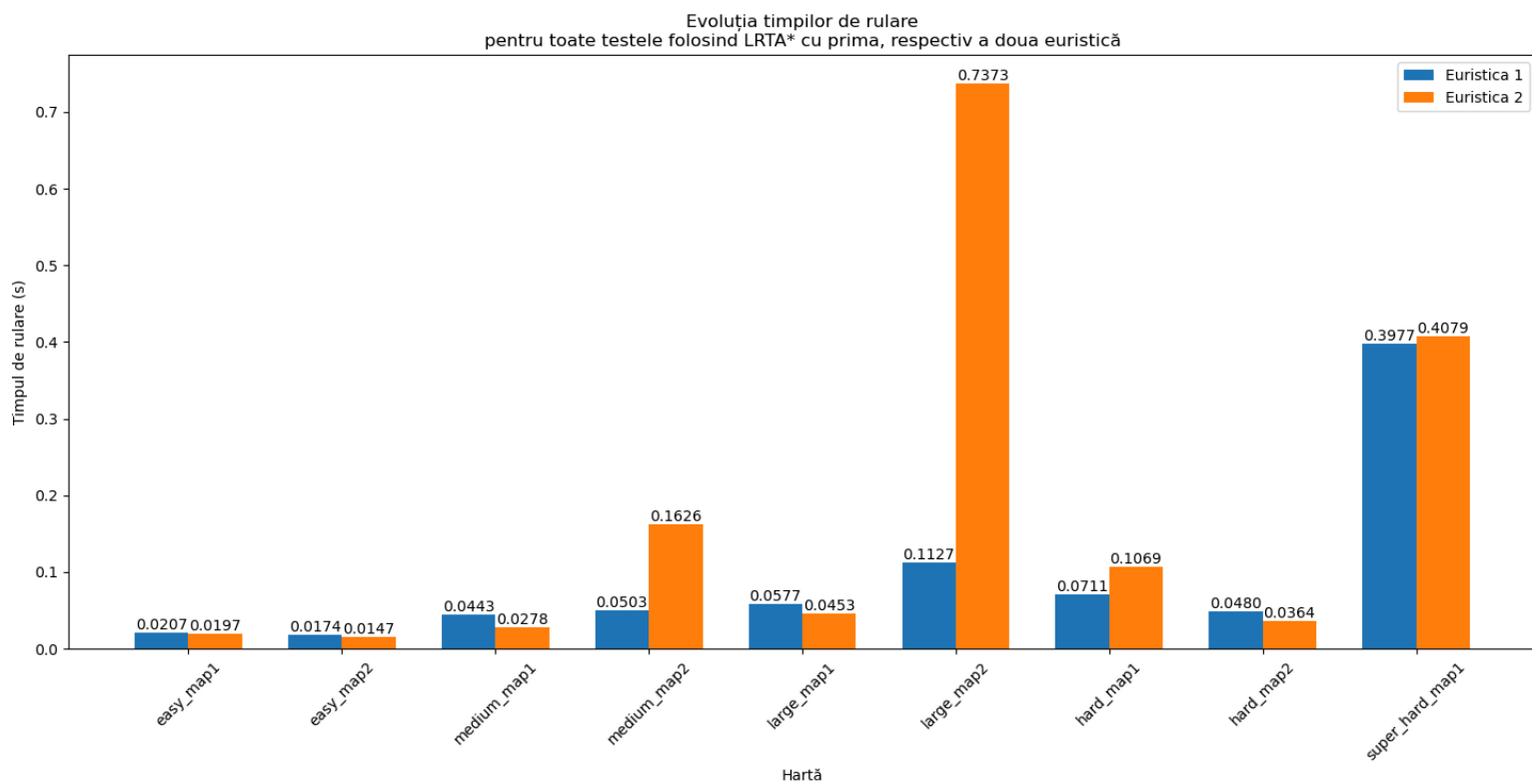
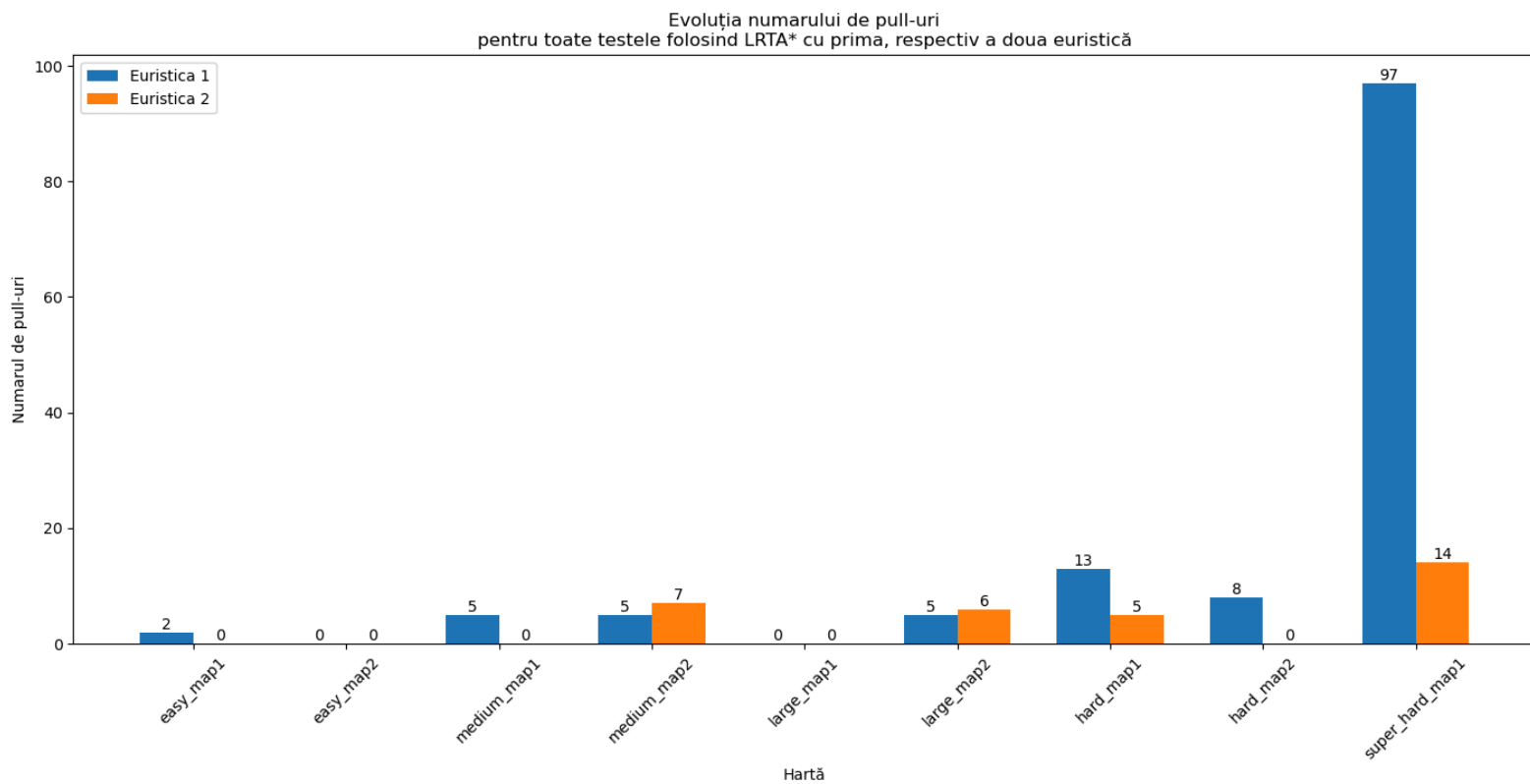
Pentru a observa cantitativ evolutia celor 2 euristici construite, am pregatit mai jos 3 grafice care ilustreaza evolutia numarului de stari explorate, al numarului de miscari de pull, respectiv al timpilor de rulare pentru algoritmul LRTA* pe harta medium_map2, trecand prin fiecare stadiu de dezvoltare al fiecarei euristici. Mai exact, am reprezentat grafic urmatoarele puncte din dezvoltarea euristiciilor:

- pentru prima euristica (h1):
 - h1 - faza1: euristica se baza doar pe suma distantelor Manhattan de la fiecare cutie la target-ul cel mai apropiat de ea, iar functia de cost penaliza stările care fusesera deja vizitate
 - h1 - faza2: Prima euristica dupa introducerea functionalitatii de evitare a deadlock-urilor
 - h1 - faza 3: Prima euristica dupa ce am tinut cont si de distanta de la player la fiecare cutie
 - h1 - final: Forma finala a primei euristici (am introdus si penalizarea pull-urilor)
- pentru a doua euristica (h2):
 - h2 - faza 1: Forma incipienta a celei de-a doua euristici, care foloseste hungarian algorithm pentru a stabili atribuirea optima a cutiilor cu target-urile si evita si blocarea in dead positions

- h2 - faza 2: A doua euristică după adăugarea distanței de la player la cea mai apropiată cutie de el
- h2 - final: Forma finală a euristicii 2, după ce am modificat funcția de cost în așa fel încât să se penalizeze mișcările de pull







Din grafice putem observa ca desi in unele cazuri numarul de stari explorate este mai mare in cazul celei de-a doua euristici, numarul de miscari de pull a fost redus fata de prima euristica.

3. Beam search

La beam search am folosit aceleasi euristici descrise anterior. Vom studia comportamentul amandurora si vom ajusta k-ul corespunzator (k = dimensiunea beam-ului). La beam search, alegerea lui k este critica: pentru un k prea mic, este posibil sa nu se gaseasca solutia pentru ca nu se exploreaza suficient spatiul de cautare, pe cand alegerea unui k prea mare va creste costurile computationale (mai multa memorie consumata, timp de calcul prelungit), dar se reduce riscul de a rata solutii bune. De aceea, alegerea lui k trebuie sa asigure un echilibru intre performanta si precizie.

- pentru $k = 1 \Rightarrow$ Algoritmul se comporta ca un best first search algorithm
- pentru k foarte mare $\Rightarrow$ Algoritmul se comporta ca o parcurgere bfs (parcurgere pe nivel)
-

Astfel, pentru fiecare dintre cele 2 euristici am crescut din aproape in aproape valorile lui k pentru a gasi k -ul minim pentru care algoritmul Beam Search gaseste solutii pentru toate hartile.

3.1. Comportamentul primei euristici pe Beam Search

Initial, folosind euristica `first_heuristic_mahattan_distance`, am incercat sa maresc din aproape in aproape k -ul pentru a urmari evolutia rezultatelor, iar abia de la $k = 8$ am ajuns sa gasesc solutie pentru fiecare harta:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0172	19	1
easy_map2.yaml	True	0.0160	9	0
medium_map1.yaml	True	0.0202	14	1
medium_map2.yaml	True	0.0384	33	5
large_map1.yaml	True	0.1919	265	24
large_map2.yaml	True	0.1064	105	10
hard_map1.yaml	True	0.0850	100	12
hard_map2.yaml	True	0.0532	60	4
super_hard_map1.yaml	True	0.0949	92	10

Problema era faptul ca explora prea mult aceleasi stari, adica ajungea in niste minime "false" in care statea ceva timp pana sa ajunga sa iasa. De aceea, cum la `Irta*` am adaugat in functia de cost o penalizare suplimentara pentru starile pe care deja le vizitase, aici am creat un "wrapper" peste euristica (`pull_and already explored penalizing heuristic`). Scopul sau este de a penaliza starile deja vizitate proportional cu numarul de vizite al starii respective pentru a descuraja ciclarea intre aceleasi stari. De asemenea, daca numarul de pull-uri aferent starii curente a crescut fata de numarul de pull-uri aferent starii anterioare, atunci adaugam o penalizare de 100 pentru a descuraja alegerea unei stari in care s-ar face mai multe miscari de pull.

În urma acestei îmbunătățiri, se găsește soluție pentru fiecare hartă începând încă de la $k = 3$:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0201	31	1
easy_map2.yaml	True	0.0147	9	0
medium_map1.yaml	True	0.0495	105	4
medium_map2.yaml	True	0.0940	222	14
large_map1.yaml	True	0.0965	186	3
large_map2.yaml	True	0.1323	220	3
hard_map1.yaml	True	0.0732	129	11
hard_map2.yaml	True	0.0608	118	4
super_hard_map1.yaml	True	0.1563	333	19

Pentru $k = 5$ se întâmplă următorul lucru: mărim dimensiunea beam-ului, adică explorăm mai multe posibilități, ceea ce înseamnă că vom putea ajunge la soluții mai optime în schimbul unui cost computațional mai mare. Pentru $k = 5$, rezultatele arată așa:

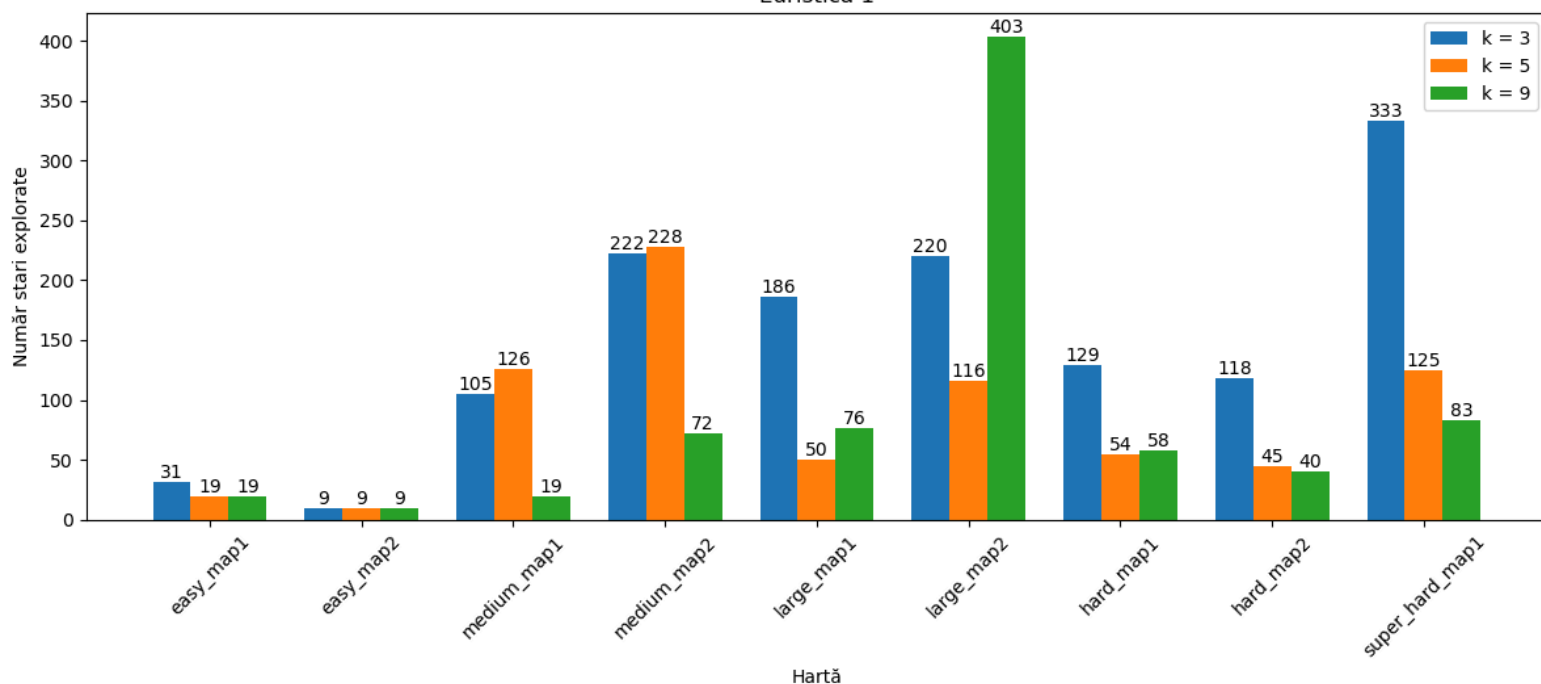
Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0207	19	0
easy_map2.yaml	True	0.0172	9	0
medium_map1.yaml	True	0.0780	126	6
medium_map2.yaml	True	0.1378	228	16
large_map1.yaml	True	0.0498	50	1
large_map2.yaml	True	0.1123	116	2
hard_map1.yaml	True	0.0516	54	3
hard_map2.yaml	True	0.0401	45	0
super_hard_map1.yaml	True	0.0990	125	5

Pentru $k = 9$:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0244	19	0
easy_map2.yaml	True	0.0180	9	0
medium_map1.yaml	True	0.0315	19	1
medium_map2.yaml	True	0.0878	72	4
large_map1.yaml	True	0.1095	76	2
large_map2.yaml	True	0.5899	403	13
hard_map1.yaml	True	0.0825	58	4
hard_map2.yaml	True	0.0536	40	1
super_hard_map1.yaml	True	0.1124	83	5

Obs: În unele cazuri, deși ne-am aștepta să obținem o soluție mai bună (mai aproape de optim) pentru un k mai mare (pentru că se pot explora mai multe căi), se poate ca, din cauza că euristica nu este chiar perfectă, să se întâmple chiar opusul, adică să ajungă să exploreze mai multe stări pentru că se blochează și este nevoit să se întoarcă la alți vecini. Cum euristica nu ține cont de obstacole, este posibil ca algoritmul să creadă că merge spre minim, dar să ajungă într-o stare blocată de obstacol și de aceea să fie de fapt un maxim.

Evoluția numărului de stări explorate
pentru toate hărțile folosind Beam Search în funcție de valoarea lui k
Euristică 1



3.2. Comportamentul celei de-a doua euristici pe Beam Search

Si in cazul acestei euristici, tot de la $k = 3$ vom ajunge sa gasim solutii pentru toate hartile.

Mapă	Solved	Timp (s)	Stări	Pull-uri

easy_map1.yaml	True	0.0206	31	1
easy_map2.yaml	True	0.0146	9	0
medium_map1.yaml	True	0.0270	33	0
medium_map2.yaml	True	0.0364	43	2
large_map1.yaml	True	0.0626	91	1
large_map2.yaml	True	0.0971	130	3
hard_map1.yaml	True	0.0640	102	6
hard_map2.yaml	True	0.0341	52	0
super_hard_map1.yaml	True	0.0499	61	3

Pentru $k = 5$, se observa, de asemenea, gasirea unor solutii mai bune decat in cazul $k = 3$, contra unui cost comutational mai ridicat.

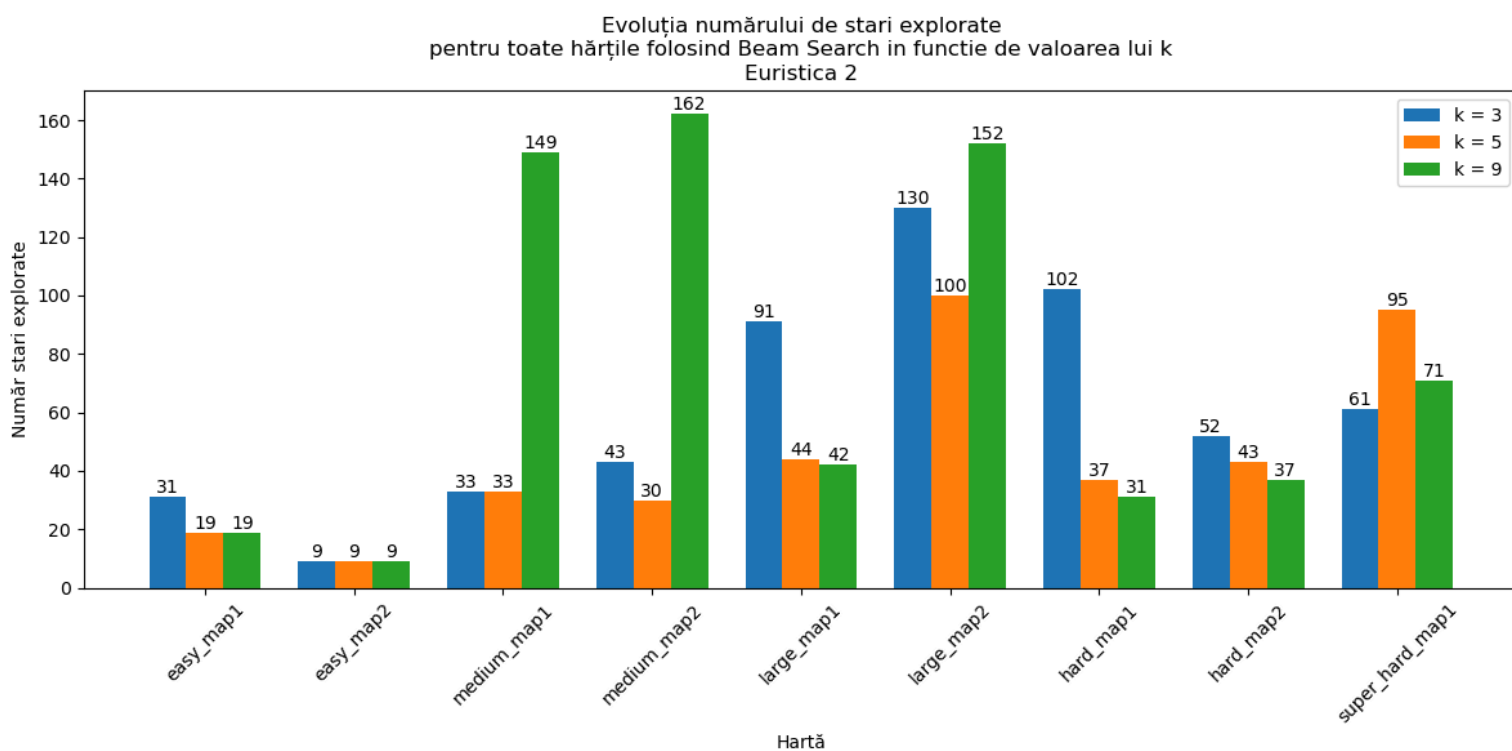
Acestea sunt rezultatele pentru $k = 5$:

Mapă	Solved	Timp (s)	Stări	Pull-uri

easy_map1.yaml	True	0.0205	19	0
easy_map2.yaml	True	0.0152	9	0
medium_map1.yaml	True	0.0354	33	0
medium_map2.yaml	True	0.0380	30	1
large_map1.yaml	True	0.0503	44	0
large_map2.yaml	True	0.1149	100	1
hard_map1.yaml	True	0.0461	37	2
hard_map2.yaml	True	0.0409	43	0
super_hard_map1.yaml	True	0.1034	95	2

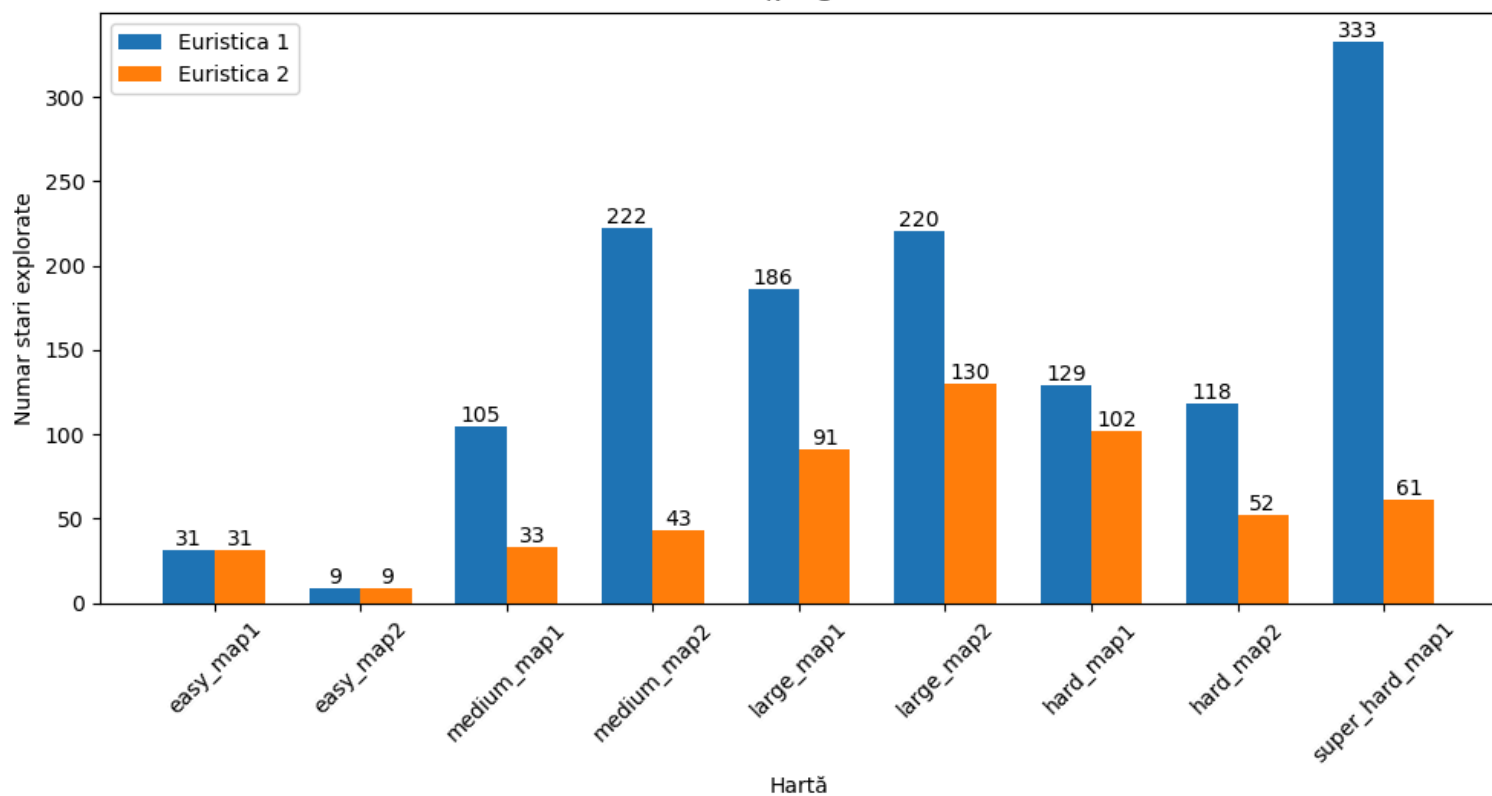
Pentru $k = 9$:

Mapă	Solved	Timp (s)	Stări	Pull-uri
easy_map1.yaml	True	0.0292	19	0
easy_map2.yaml	True	0.0188	9	0
medium_map1.yaml	True	0.1681	149	4
medium_map2.yaml	True	0.2061	162	11
large_map1.yaml	True	0.0767	42	0
large_map2.yaml	True	0.2850	152	2
hard_map1.yaml	True	0.0606	31	0
hard_map2.yaml	True	0.0542	37	1
super_hard_map1.yaml	True	0.1292	71	3

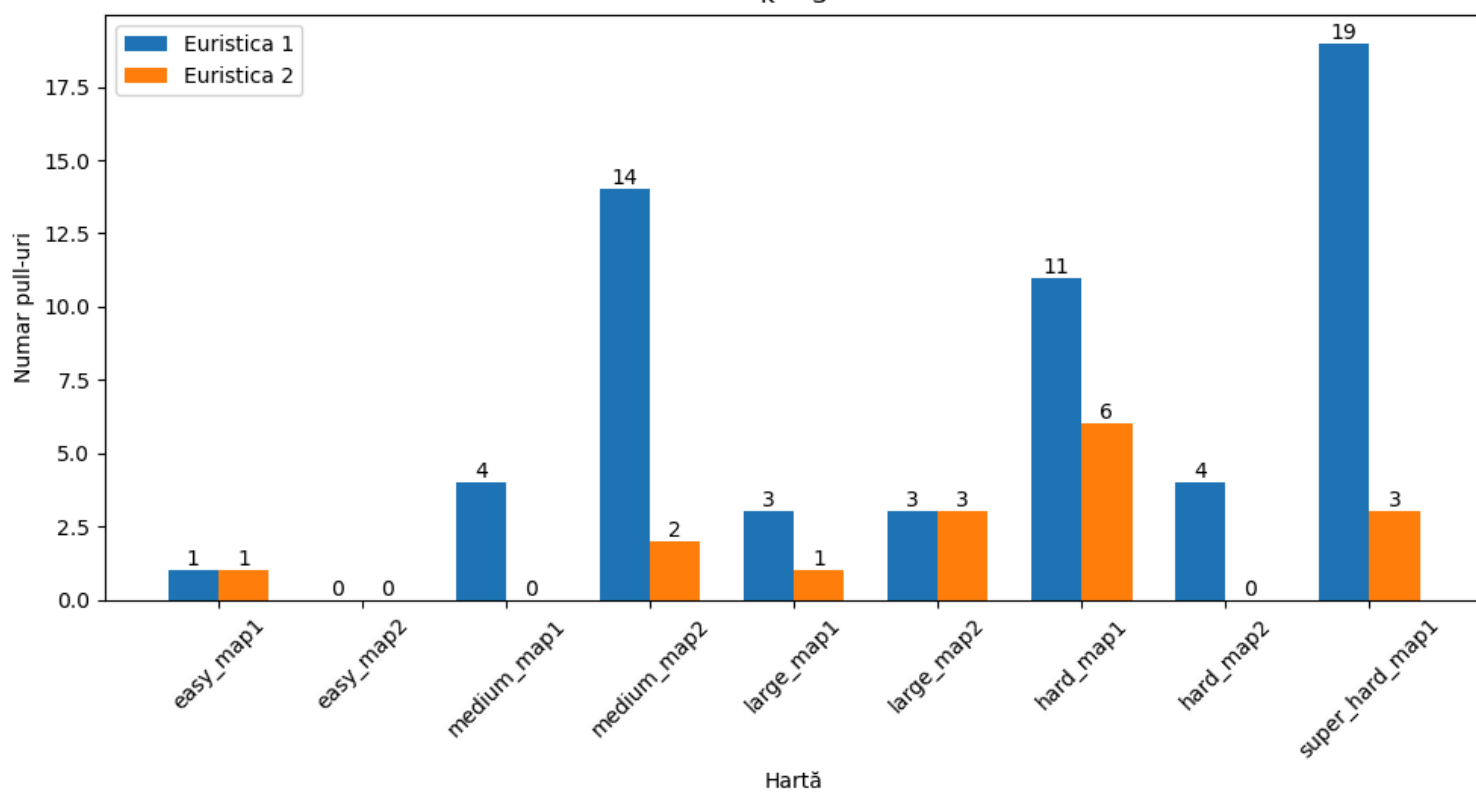


Analiza cantitativă a rezultatelor algoritmului Beam Search pentru $k = 3$ comparând cele 2 euristici:

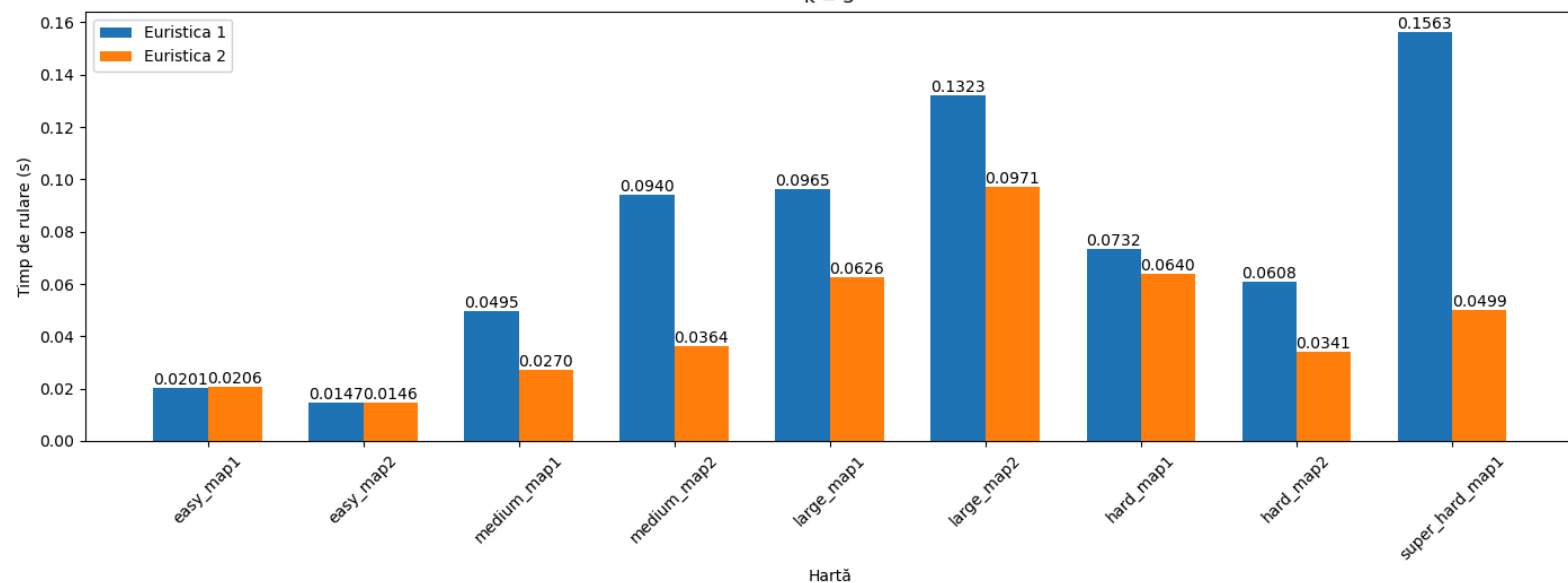
Evoluția numărului de stări explorate
pentru toate hartile folosind Beam Search cu prima, respectiv a doua euristică
 $k = 3$



Evoluția numărului de pull-uri
pentru toate hartile folosind Beam Search cu prima, respectiv a doua euristică
 $k = 3$



Evoluția timpilor de rulare
pentru toate hartile folosind Beam Search cu prima, respectiv a doua euristică
k = 3



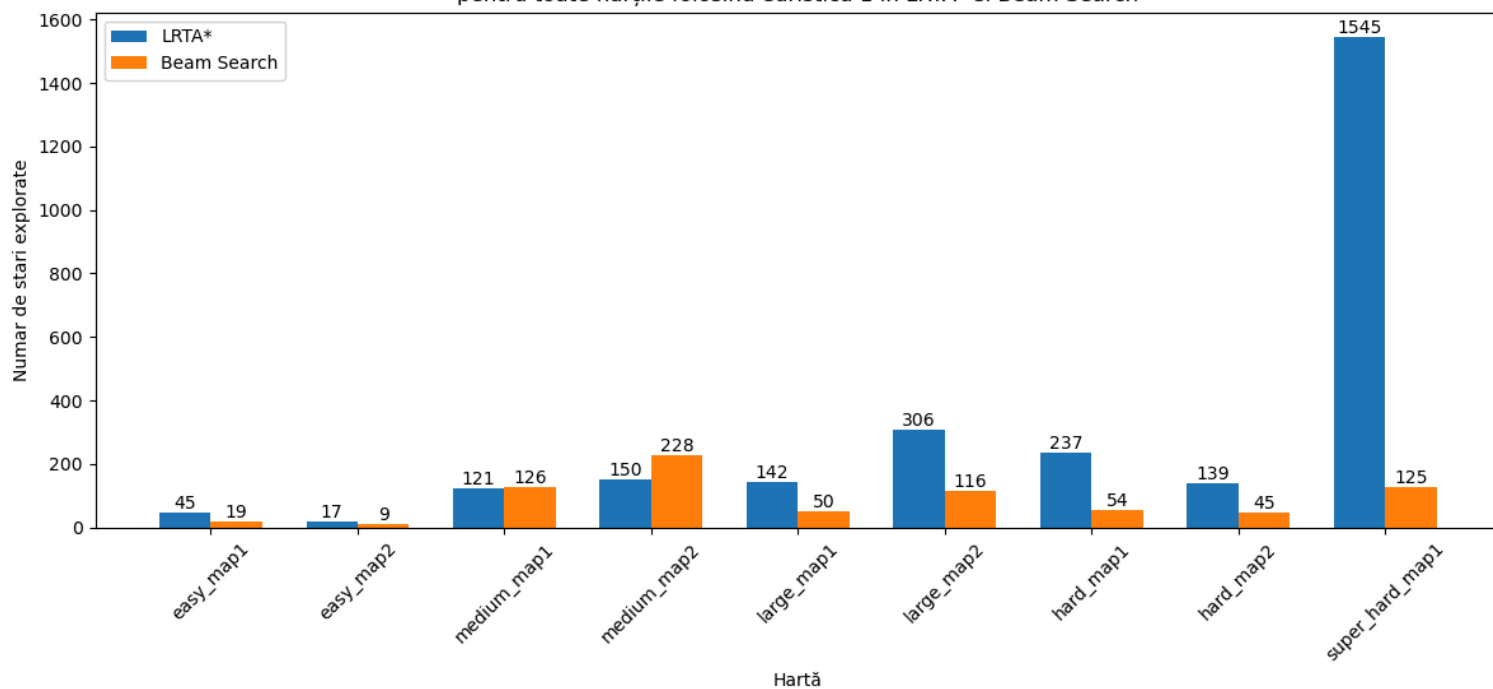
Urmărind aceste grafice comparative, putem trage concluzia ca algoritmul Beam Search ajunge mai repede la soluție folosind a doua euristică, soluția este mai bună, căci numărul de stări explorate este mai mic și numărul de pull-uri este mai mic.

4. Concluzii - Comparatie rezultate Lrta* - beam search

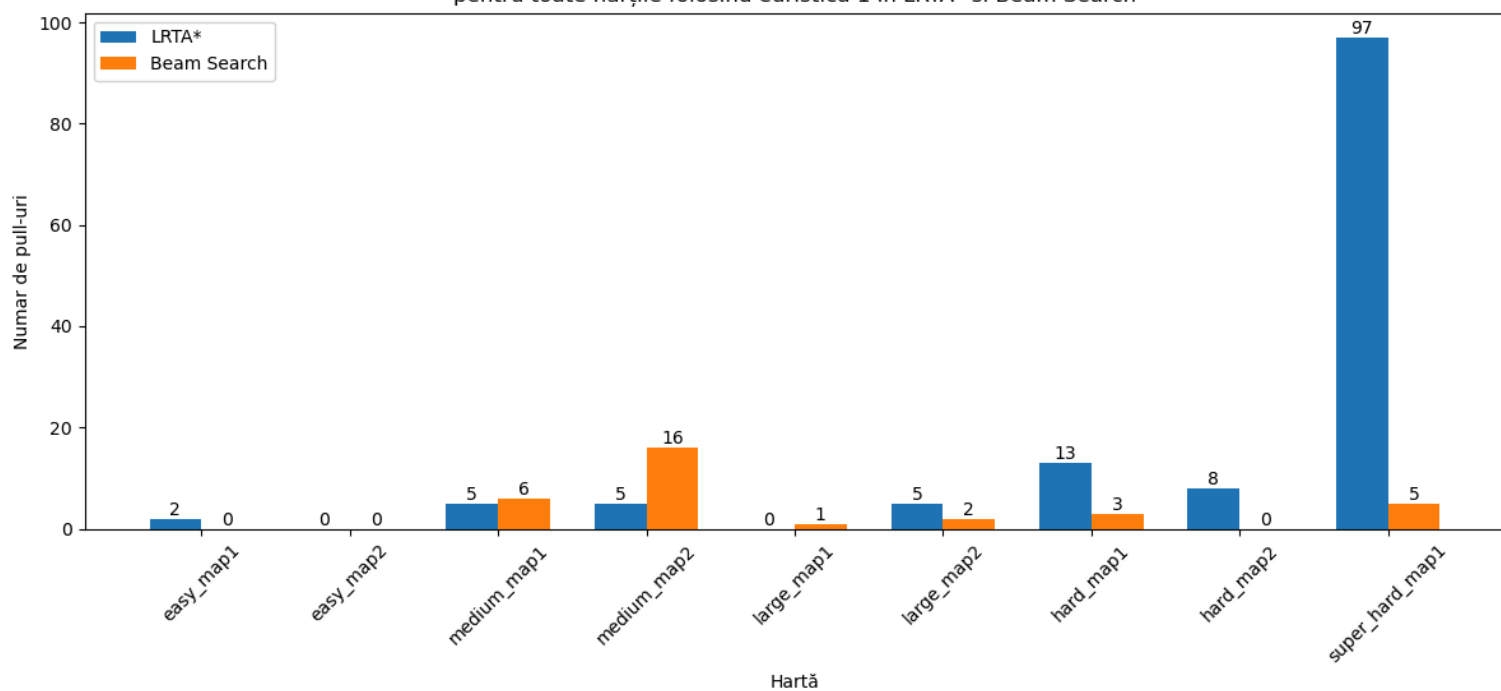
Obs: La Beam Search, am considerat rezultatele corespunzătoare lui k = 5.

Pentru prima euristică:

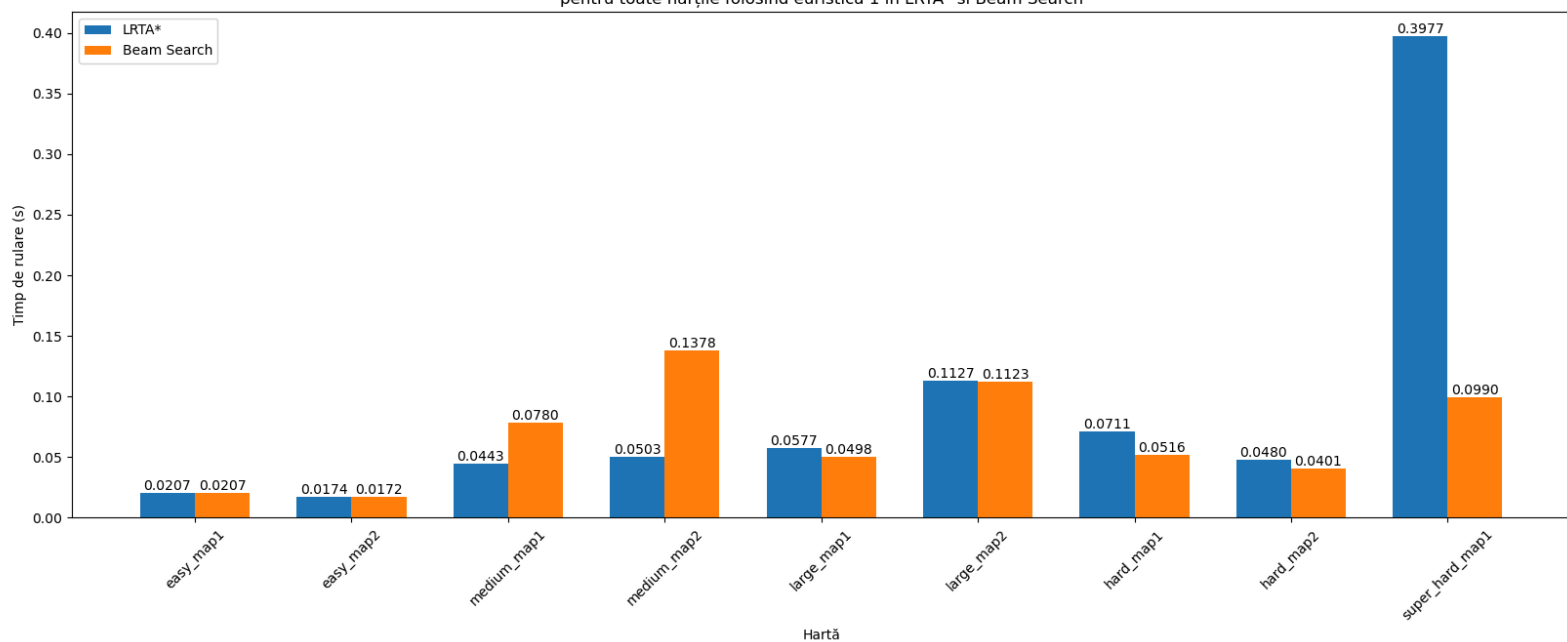
Evoluția numărului de stări explorate
pentru toate hărțile folosind euristică 1 în LRTA* și Beam Search



Evoluția numărului de pull-uri
pentru toate hărțile folosind euristica 1 în LRTA* și Beam Search

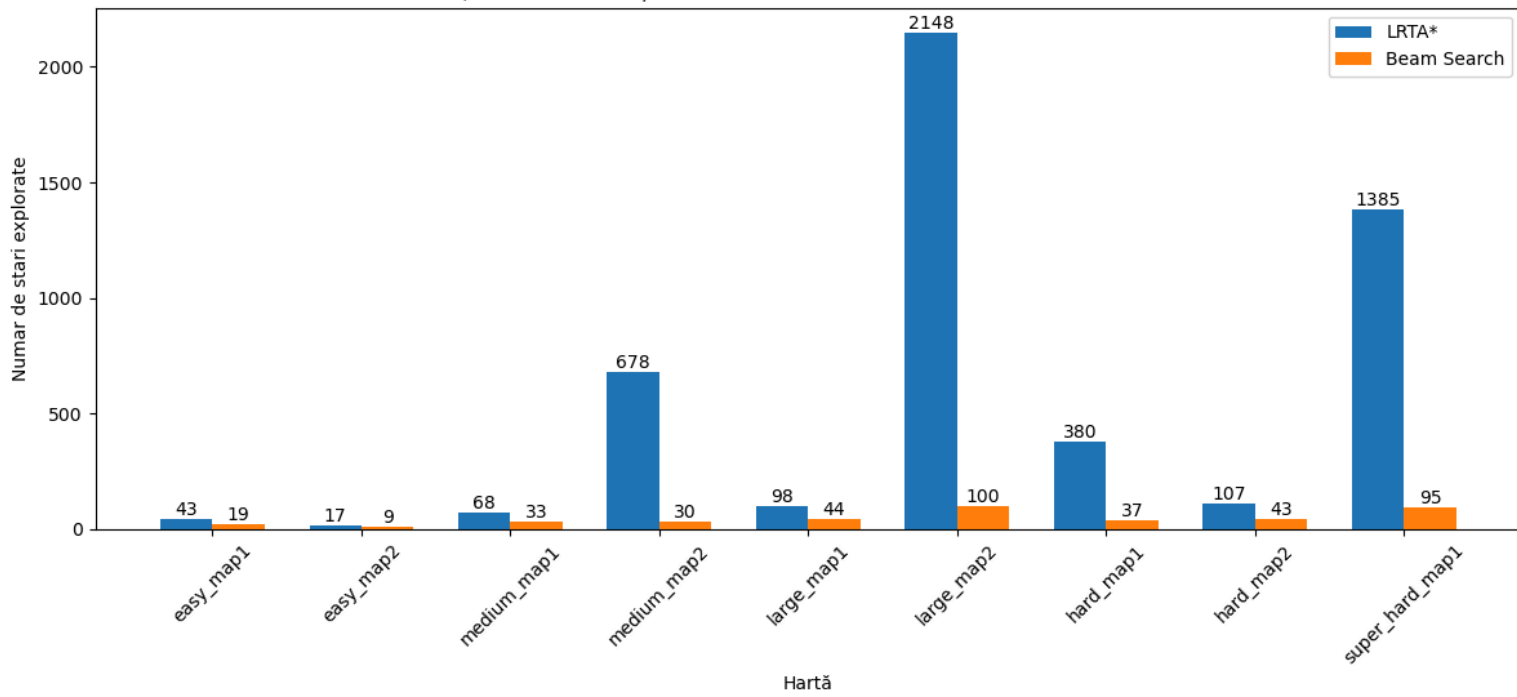


Evoluția timpilor de rulare
pentru toate hărțile folosind euristica 1 în LRTA* și Beam Search

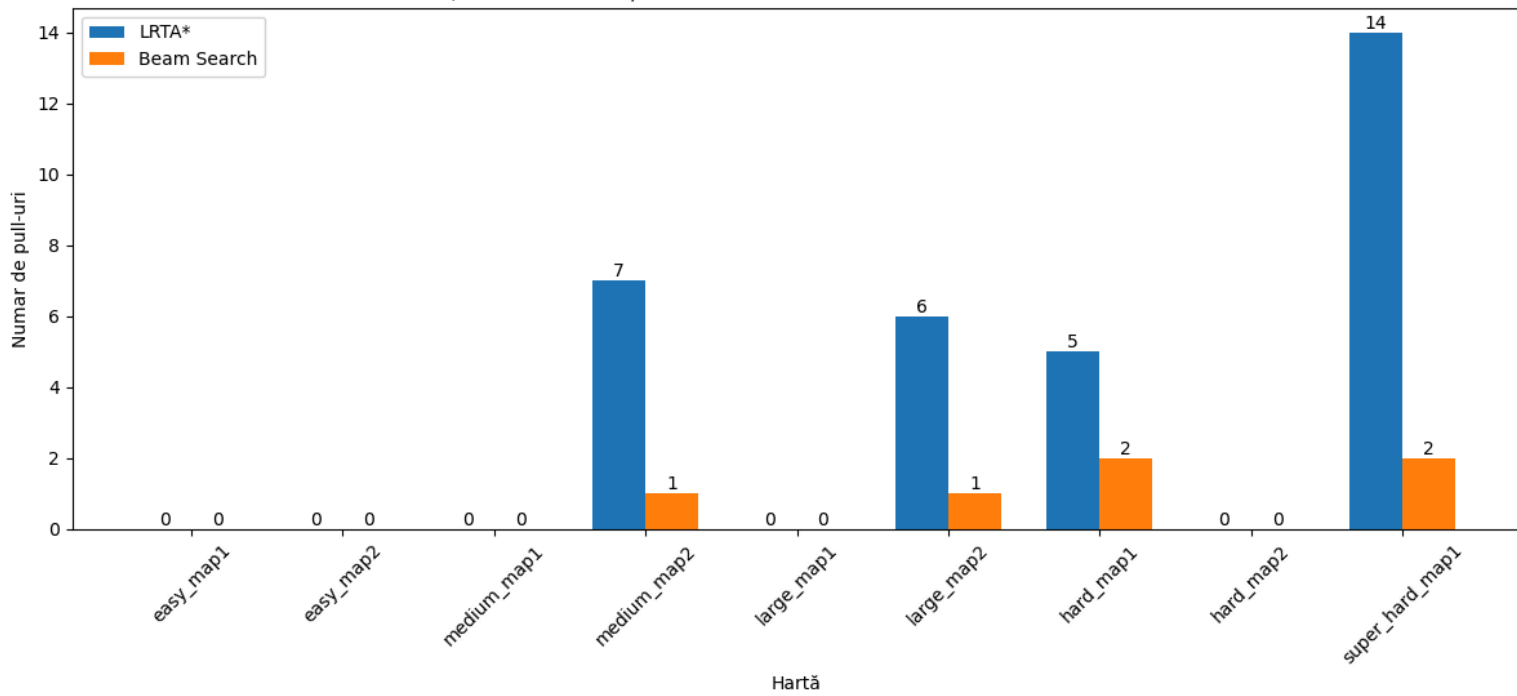


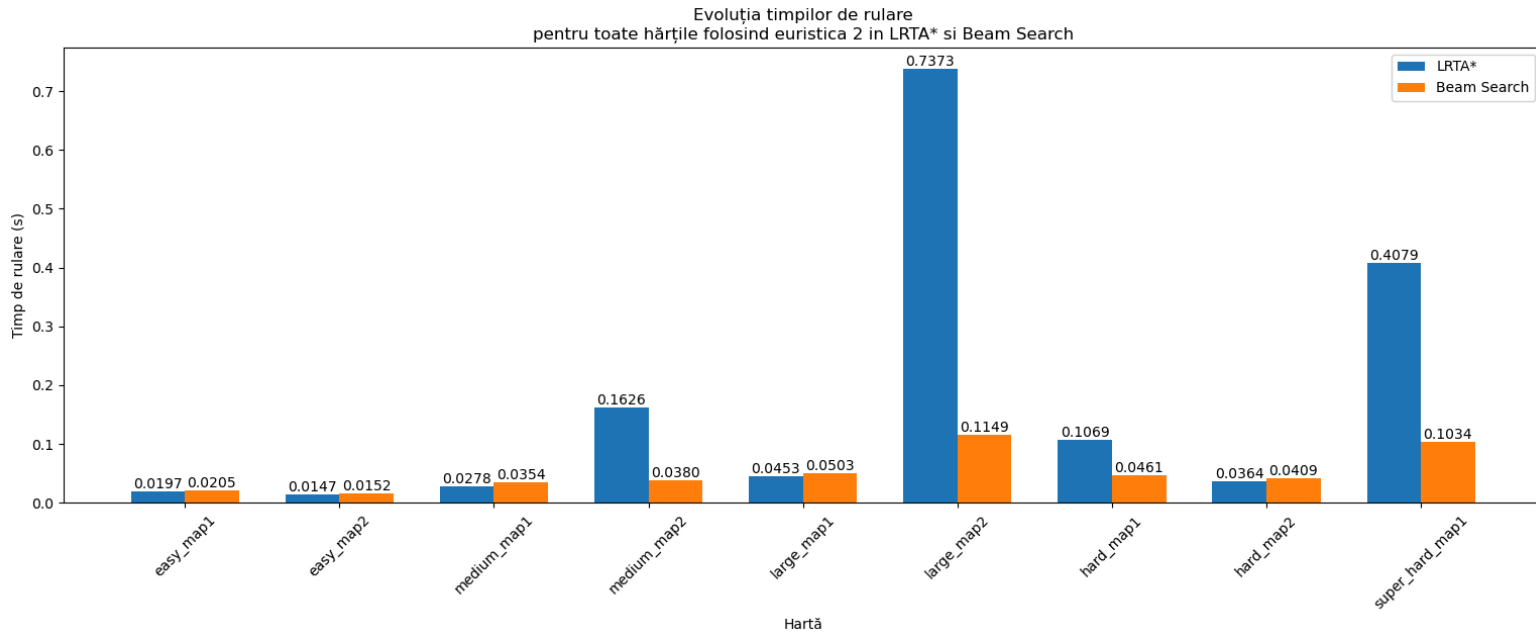
Pentru a doua euristică:

Evoluția numărului de stări explorate
pentru toate hărțile folosind euristică 2 în LRTA* și Beam Search



Evoluția numărului de pull-uri
pentru toate hărțile folosind euristică 2 în LRTA* și Beam Search





Ca principiu de baza, LRTA* caută soluția pe baza estimării curente a distanței de la starea în care se afla la goal, iar, în același timp, învață din experiența mișcărilor anterioare, pe când Beam Search păstrează la fiecare pas cele mai promitatoare k stări candidate, urmând să le exploreze doar pe acestea. De aceea, calitatea euristicii face o diferență mai mare aici decât la LRTA*, căci beam search-ul explorează doar un subset de k stări la fiecare pas, ceea ce înseamnă că ordonarea dată de euristica trebuie să fie foarte bună pentru că altfel putem rata complet o cale care ne-ar putea duce la soluție. Spre deosebire de beam search, LRTA*-ul învață și actualizează la fiecare pas valorile $h(n)$, este supus unui proces dinamic, care corectează în timp estimările, euristica fiind doar un punct de plecare, căci LRTA*-ul are capacitate de “auto-corectare”.

După cum se observă și din graficele comparative între cei doi algoritmi, LRTA*-ul consumă mai mult timp decât Beam Search-ul (LRTA*-ul devine mai eficient abia pe măsura ce continuă să ruleze pe aceeași hartă, întrucât își îmbunătățește estimările și învață soluția optimă în timp). Pe de altă parte, LRTA*-ul explorează mai multe căi posibile și poate găsi soluții mai bune pe termen lung. Într-un spațiu de căutare finit, în care costurile sunt pozitive, în care există o cale de la orice stare la starea țintă și în care se utilizează estimări admisibile nenegative, LRTA*-ul este chiar complet, adică se garantează că va ajunge mereu la soluție. Spre deosebire de acesta, Beam search-ul risca să rămână blocat în minime locale, dar are avantajul faptului că este mai rapid, fiind eficient în explorarea celor mai promitatoare căi pe termen scurt. Pentru a evita problema blocării în minime locale, am introdus o penalizare a stărilor deja vizitate, proporțională cu numărul de vizite al stării respective. Un alt factor critic al beam search-ului este alegerea k -ului (beam width), întrucât un k prea mic ar putea duce la pierderea unor soluții (pentru că nu permite explorarea suficientă a spațiului de căutare), iar un k prea mare ar putea fi costisitor din punct de vedere computațional. Am observat că pentru un k prea mare la hărțile mici, timpul de rulare crește, fără a duce la o soluție mai bună, iar în cazul hărților mari, pentru un k prea mic se poate să nu ajungem la nicio soluție.

Resurse:

[1]: <https://www.geeksforgeeks.org/what-is-reinforcement-learning/>

[2]:

<https://era.library.ualberta.ca/items/51ae0a8b-07a5-48ea-b2fc-02412120c622/view/5f43fe16-25a7-43c1-a163-f6a2c9defe66/NQ46861.pdf>

[3]: <https://weetu.net/Timo-Virkkala-Solving-Sokoban-Masters-Thesis.pdf>

[4]: <https://brilliant.org/wiki/hungarian-matching/>

[5]:

https://docs.scipy.org/doc/scipy-0.18.1/reference/generated/scipy.optimize.linear_sum_assignment.html

[6]:

<https://hussainwali.medium.com/simple-implementation-of-beam-search-in-python-64b2d3e2fd7e>